Regresión logística y clasificación

Segunda herramienta para el gerente: un aviso de riesgo de cancelación para decidir cuándo vender por encima de la capacidad (*overbooking*). Predecir una **categoría**.

De una suma ponderada a una probabilidad

`probabilidad = 1 / (1 + e^(−z))`: la **sigmoide** comprime la suma ponderada `z` entre 0 y 1.

```
logit = Pipeline([("prep", prep), ("clf", LogisticRegression(max_iter=1000))])   # Ejercicio 1
logit.fit(X_train, y_train)
proba_test = logit.predict_proba(X_test)[:, 1]
```

- `predict_proba` da la probabilidad; `predict` aplica un **umbral de 0,5** por defecto.
- Todo el preprocesado va dentro del `Pipeline` (como en el Sprint 6).

Interpretar: *odds ratios*

- *Odds* = P(cancela) / P(no cancela). `exp(coeficiente)` = **odds ratio**: cuánto se **multiplican** los odds.
- > 1 aumenta el riesgo; < 1 lo reduce.

```
odds = pd.Series(np.exp(coefs), index=nombres)                    # Ejercicio 2
or_no_reembolsable = odds["cat__tarifa_no_reembolsable"]
or_deposito_total  = odds["cat__deposito_total"]
mas_riesgo = odds.idxmax()
```

Ojo: que la tarifa no reembolsable tenga un odds ratio < 1 **no significa que cause** menos cancelaciones; puede ser que quien la elige ya es más seguro.

El umbral y la matriz de confusión

	Predicho: se mantiene	Predicho: cancela
Real: se mantiene	Verdadero negativo	**Falsa alarma (FP)**
Real: cancela	**Cancelación no prevista (FN)**	Verdadero positivo

- El modelo da una probabilidad; **nosotros** decidimos desde qué valor hablamos de «riesgo».
- Ejercicio 3: con `metricas_umbral(y_test, proba_test, umbral)`, compara 0,5 y 0,3: **sube el recall** (`sube_recall`) y **baja la precisión** (`baja_precision`).

Elegir el umbral según el coste

- **FN**: habitación vacía que podíamos haber vendido. **FP**: molestamos a un cliente que iba a venir o hacemos *overbooking* sin necesidad.
- Si un FN cuesta el **triple** que un FP, conviene ser más «alarmista».

```
umbrales = np.arange(0.05, 0.96, 0.05)
costes = [coste_total(y_train, proba_cv, u) for u in umbrales]   # FN = 3, FP = 1  (Ejercicio 4)
mejor_umbral = umbrales[int(np.argmin(costes))]
```

Mini-reto: `tabla_umbrales` con 0,5, 0,3 y `mejor_umbral` (recall, precisión y coste **en test**) y una `recomendacion` con al menos una cifra. **El gerente no quiere oír «AUC»: quiere euros.**

Datos desbalanceados (cuaderno S07_04)

```
tonto = DummyClassifier(strategy="most_frequent").fit(X_train, y_train)   # Ejercicio 1
acc_tonto, recall_tonto = tonto.score(X_test, y_test), recall_score(y_test, tonto.predict(X_test))
```

> La **paradoja de la exactitud**: un modelo que nunca avisa de una cancelación acierta mucho… y tiene recall 0.

Tres remedios: **pesos por clase** (`class_weight="balanced"`), **submuestreo** de la clase mayoritaria y **ajustar el umbral**.

ROC frente a PR

Curva ROC y AUC

TPR (recall) frente a FPR para cada umbral. Diagonal = modelo inútil. `roc_curve`, `roc_auc_score` (Ejercicio 2).

Curva PR y AP

Precisión frente a recall, solo con la clase que importa. Más honesta con clases raras. `average_precision_score` (Ejercicio 3).

- Con clases raras la ROC puede parecer excelente aunque **la mayoría de las alarmas sean falsas**.
- La línea de referencia de la PR es la **prevalencia**: `veces_mejor_que_azar = ap_base / prevalencia`.

Remedios y umbral con restricción

```
nuevo_modelo(class_weight="balanced").fit(X_train, y_train)      # Ejercicio 4
resample(negativas, replace=False, n_samples=2 * len(positivas),
         random_state=SEMILLA)                                     # Ejercicio 5 (solo en train)

prec, rec, umbrales = precision_recall_curve(y_train, proba_cv)    # Ejercicio 6
umbral_70 = umbrales[rec[:-1] >= 0.70].max()                       # detectar ≥ 70 %
```

- Remuestrear **solo el entrenamiento**; el test se deja como es en la realidad.
- Mini-reto: `comparativa` de `"base"`, `"pesos"` y `"submuestreo"` con AUC, AP, recall y precisión en test.

Ahora, a practicar

Lo que viene a continuación está en tu repositorio; no hace falta buscar ejercicios fuera.

CUADERNO

S07_03_regresion_logistica — 55 min

Regresión logística

sprints/S07/notebooks/S07_03_regresion_logistica.ipynb

RETO

Reto del sprint — 30 min

Acordar las hipótesis de coste y hacer la regresión del precio (pasos 1–2)

sprints/S07/guia/Reto_Sprint_07.md

CUADERNO

S07_04_roc_pr_desbalance — 40 min

ROC, PR y datos desbalanceados

sprints/S07/notebooks/S07_04_roc_pr_desbalance.ipynb